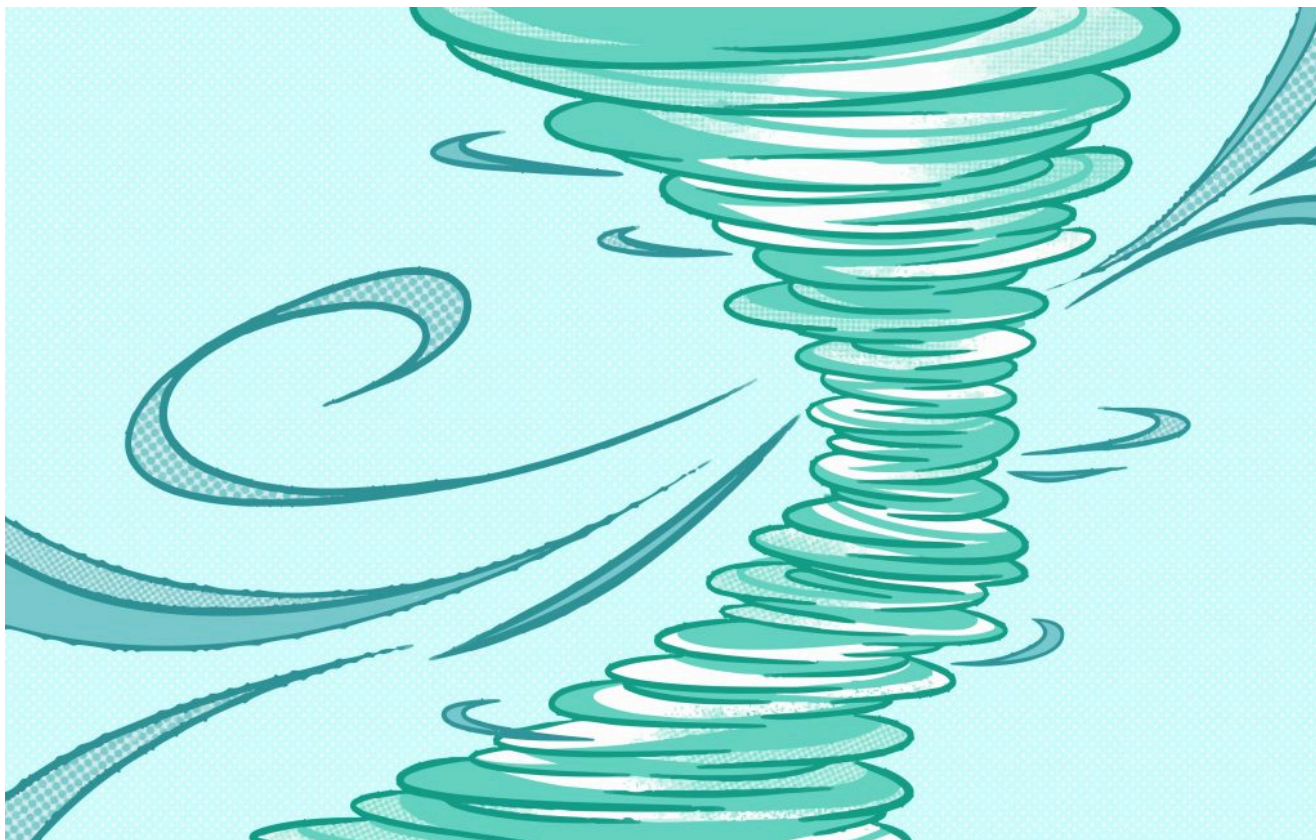


ЭКОСИСТЕМА CLOUD NATIVE / БАЗЫ ДАННЫХ / НАБЛЮДАЕМОСТЬ

Как Jaeger достиг сжатия в 8,6 раза при обработке 10 миллионов диапазонов с помощью ClickHouse

В Jaeger v2.18.0 добавлена встроенная поддержка ClickHouse, обеспечивающая коэффициент сжатия 8,6 и сверхбыстрые запросы к 10 миллионам диапазонов.

24 мая 2026 года, 11:00 [Махад Заряб](#)



Кёнхи Им для Unsplash+

[CNCF](#) спонсировала этот пост.

Как человек, занимающийся поддержкой **Jaeger**, я видел, что пользователи постоянно запрашивали поддержку **ClickHouse** на протяжении последних нескольких лет. В Jaeger версии 2.18.0 мы наконец-то реализовали эту возможность. Больше всего меня радует не только то, что ClickHouse

записываемых только в конец, и выполняет сложные аналитические агрегации за миллисекунды, предоставляя командам высокоэффективное хранилище производственного уровня.

Для новичков в проекте Jaeger - это специализированная платформа **распределенной трассировки** Cloud Native Computing Foundation (CNCF), созданная для мониторинга сложных микросервисов и устранения неполадок. Он отслеживает запросы через границы службы, чтобы выявить узкие места с задержкой и первопричины, что в конечном итоге сокращает среднее время восстановления команды (MTTR). Изначально интегрировав ClickHouse, Jaeger теперь может использовать столбчатое хранилище для обеспечения невероятно высокой производительности запросов и сжатия данных с высоким коэффициентом сжатия на миллиардах сегментов.

В этом посте я расскажу, почему **ClickHouse** — отличный выбор для хранения трассировок, как устроена схема и как начать использовать ее с Jaeger уже сегодня.

Почему выигрывает колоночное хранилище

По сути, проблема трассировки заключается в двух аспектах: хранении огромных объемов полуструктурированных данных о событиях и последующем быстром поиске этих данных по нескольким параметрам: сервису, операции, тегам, продолжительности, временному диапазону и идентификатору трассировки. Cassandra и Elasticsearch хорошо зарекомендовали себя в сообществе Jaeger, но их использование сопряжено с операционными расходами. Индексирование увеличивает задержку и затраты. Масштабирование становится сложной задачей. Решения о хранении данных вынуждают идти на болезненные компромиссы.

Высокая пропускная способность и запросы с низкой задержкой

ClickHouse — это колоночно-ориентированная **OLAP-база данных**, разработанная именно с учетом этих ограничений: высокая пропускная способность при приеме данных, агрессивное сжатие и быстрые аналитические запросы. Для трассировки это практически идеальный вариант. Данные трассировки по своей природе повторяются: одни и те же названия сервисов, операций, коды состояний и теги встречаются снова и



The Cloud Native Computing Foundation (CNCF) hosts critical components of the global technology infrastructure including Kubernetes, OpenTelemetry, and Argo. CNCF is the neutral home for cloud native collaboration, bringing together the industry's top developers, end users, and vendors.

[Learn More →](#)

THE LATEST FROM CNCF

[Building a Cluster-Aware AI Agent with Kubernetes, Argo CD, and GitOps](#)

25 June 2026

[From Awareness to Engineered Accessibility in Open Source](#)

24 June 2026

[Building Jaeger's ClickHouse backend: 8.6× compression on 10 million spans](#)

23 June 2026

"Trace data is repetitive by nature—the same service names, operation names, status codes, and tags appear over and over. A columnar layout thrives on that repetition."

Compression that actually matters

We measured significant compression gains on trace data. Service names like "auth-service" or "payment-gateway" appear hundreds of thousands of times. Same with operation names, tag keys, and status codes. In a row-oriented database, that redundancy goes uncompressed. In a column-oriented one, ClickHouse groups identical values, making them trivial to compress. The result? An 8.6× compression ratio on the spans table in our benchmarks.

I-time analytics

includes native ClickHouse SQL methods to directly compute service-level latency, call rates, and error rates from your stored spans. This allows teams to generate core health and performance metrics for their microservices straight from their trace data, without needing an external metrics pipeline.

TRENDING STORIES

1. [LLMs create a new blind spot in observability](#)
2. [Observability overload is drowning engineers](#)
3. [After becoming cloud computing's telemetry standard, OpenTelemetry graduates into the AI infrastructure era](#)
4. [How AI Can Help IT Teams Find the Signals in Alert Noise](#)
5. [Vendor neutrality isn't magic: A hard look at the OpenTelemetry ecosystem](#)

Designing the schema

Schema design was where things got tricky. We needed to optimize for Jaeger's core query patterns: trace lookup by trace ID, service, and operation; attribute filtering; time-range queries; and the aggregation powering the [Service Performance Monitoring \(SPM\)](#) feature. These constraints don't all pull in the same direction.

There's an excellent earlier [post](#) by Ha Anh Vu that benchmarked ClickHouse schemas for Jaeger v1, and that work laid the foundation. However, Jaeger v2 adopts the OpenTelemetry data model, which forces us to revisit several decisions.

The design space is documented in detail in an [Architectural Decision Record \(ADR\)](#). The sections below walk through some of the key decisions worth understanding.

Trade-offs in primary key

In ClickHouse, the primary key isn't a uniqueness constraint. Instead, it defines the on-disk sort order and powers a sparse index (one index per 8,192-row granule). Making it is the single highest-leverage decision in the schema.

1. **Optimize for trace retrieval:** sort by `trace_id`. Every span of a trace lands in one contiguous block, so `GetTrace` is a single seek + sequential read. However, search queries pay for this optimization, as the `service_name` and `operation_name` filters cannot use the primary key index at all.
2. **Optimize for search (chosen):** sort by `(service_name, name, start_time)`. Search queries that filter by service, operation, and a time window become direct primary-key lookups.

The decision came down to an asymmetric trade-off. Sorting by `trace_id` makes search performance terrible, but sorting by `(service_name, name, start_time)` hurts trace retrieval much less, because we can recover most of the lost performance with two cheap mechanisms:

1. A `bloom_filter` skip index on `trace_id`, which lets the engine prove a granule can't contain a given ID without reading it.
2. A `trace_id_timestamps` materialized view that tells the search path each matching trace's time bounds, so the follow-up `GetTraces` call can prune partitions and granules.

An earlier benchmark run with the schema sorted by `trace_id` clearly showed the asymmetry. Trace retrieval was about 27 ms, but a search query took about 880 ms. Re-sorting by `(service_name, name, start_time)` pushed trace retrieval to around 100 ms (slower, but still well under interactive thresholds) while bringing multi-filter search down to about 140 ms.

Storing typed attributes

In Jaeger v1, tags were always strings. The v2 reader API accepts a typed map, where attributes can be `Bool`, `Int64`, `Float64`, `String`, or one of the complex types (`Bytes`, `Slice`, `Map`). We need to query across these types, so the storage layer can't collapse everything to strings.

The schema leverages ClickHouse's **Nested** column per primitive type, repeated at the span, event, link, resource, and scope level. Think of it as a mini table inside each row; each can have its own set of attribute names and values. This approach lets attribute filters use the same query semantics as querying a regular table.

However, it is worth noting that Attribute-only searches are inherently more expensive because they cannot fully leverage ClickHouse's primary index. The engine's index is optimized around top-level structural fields—specifically `service`,

limit the data ClickHouse has to scan.

Materialized views

Some of Jaeger's queries don't fit the spans table's sort order. For example, the Jaeger UI needs to quickly load the full list of known service names and operations, while trace searches often need efficient access to trace time ranges.

Rather than answering these with expensive table scans, we use materialized views to precompute the data. In ClickHouse, **materialized views** automatically transform inserts into a source table and write the results into optimized target tables.

This approach is used to speed up queries for service names, operations, and trace ID timestamp ranges.

Five levels of attributes

A technical challenge that may not be immediately obvious from the span's schema: how the storage layer interprets attribute lookups. For instance, when searching for `http.status_code=200`, the system cannot inherently distinguish if "200" is a string, an integer, a span-level attribute, or a resource-level attribute. Depending on the service, the same logical key could be categorized under `str_attributes` or `int_attributes`, and it might exist at any of the five data levels: resource, scope, span, event, or link.

To solve this, we maintain a dedicated **attribute_metadata** table, populated by materialized views off the spans table. This allows the reader to look up the filter key at query time and only query the columns for the types and levels that were observed.

Span throughput at scale

We benchmarked the ClickHouse backend using 10 million spans across 1 million traces on a single-node deployment. The benchmark measured ingestion throughput, compression, trace retrieval, and filtered search latency.

The backend sustained more than 50k spans/sec during ingestion, achieved an 8.6× compression ratio on the spans table, and reduced span data by nearly 6 GiB to roughly 722 MiB on disk. Trace retrieval averaged around 100 ms, while most

“The backend sustained more than 50k spans/sec during ingestion, achieved an 8.6× compression ratio on the spans table, and reduced span data by nearly 6 GiB to roughly 722 MiB on disk.”

These numbers are encouraging, but they should be read in the context of the benchmark environment and dataset. Full methodology, configuration, and query details are available in the [benchmarking report](#).

Getting started

ClickHouse support is available in alpha as a storage backend starting with Jaeger v2.18.0. You'll need a running ClickHouse instance and the Jaeger v2 configuration for the ClickHouse backend. The full instructions are described in the [setup guide](#).

Being a Jaeger maintainer has been one of the most rewarding parts of my career so far. If you want to chat about this work, contribute, or report issues, please open one on [GitHub](#) or find us in the CNCF #jaeger Slack.

TNS



Mahad Zaryab is a Software Engineer at Meta on the Zero Trust Networking team. Previously, he worked as a Software Engineer at Bloomberg. Mahad attended the University of Waterloo where he got a degree in Engineering. For over a year,...

[Read more from Mahad Zaryab →](#)

TNS owner Insight Partners is an investor in: ClickHouse, Slice, Real.

SHARE THIS STORY



TRENDING STORIES

.LMs create a new blind spot in observability

graduates into the AI infrastructure era

4. How AI Can Help IT Teams Find the Signals in Alert Noise
5. Vendor neutrality isn't magic: A hard look at the OpenTelemetry ecosystem

INSIGHTS FROM OUR SPONSOR



The Cloud Native Computing Foundation (CNCF) hosts critical components of the global technology infrastructure including Kubernetes, OpenTelemetry, and Argo. CNCF is the neutral home for cloud native collaboration, bringing together the industry's top developers, end users, and vendors.

[Learn More →](#)

From Awareness to Engineered Accessibility in Open Source

24 June 2026

Building Jaeger's ClickHouse backend: 8.6× compression on 10 million spans

23 June 2026

Agent Auth: A lawyer's day in court

23 June 2026

Telemetry that matters: Designing sustainable, high-impact observability pipelines

22 June 2026

KubeCon + CloudNativeCon, OpenInfra Summit and PyTorch Conference Unite in China to Scale AI

18 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)